STAT 545 HW 10

Bowen Zheng

2026-04-15

Problem 1

Consider the group lasso penalty for high-dimensional regression problems. Suppose $Y \in R^n$ and $X \in R^{n \times p}$. The group lasso proposes to divide the p predictors into L groups where p_l is the number of predictors in group l and $\sum_{l=1}^{L} p_l = p$. Then the group lasso solves the following optimization problem:

$$argmin_\beta(\frac{1}{2}||Y - \sum_{l=1}^{L} X_l\beta_l||_2^2 + \lambda \sum_{l=1}^{L} \sqrt{p_l}||\beta_l||_2)$$

where $||\beta_l||_2 = (\beta_l^T\beta_l)^{1/2}$, the Euclidean norm for β_l and λ is a tuning parameter. Assume $X_l^T X_l = I$ for all l and consider a coordinate descent approach to solve for l. Show that when the solution β_l is non-zero, it is given by

$$\hat{\beta}_l = (1 - \frac{\lambda\sqrt{p_l}}{||S_l||_2})S_l$$

where $S_l = X_l^T(Y - \sum_{k=1,k\neq l}^{L} X_k\beta_k)$.

In this case, we are searching for β such that we minimize $f(\beta) = \frac{1}{2}\|Y - \sum_{l=1}^{L} X_l\beta_l\|_2^2 + \lambda\sum_{l=1}^{L}\sqrt{p_l}\|\beta_l\|_2$. We see that S_l contains a the partial residuals of group l. We denote the partial residuals as $R_l = Y - \sum_{k\neq l} X_k\beta_k$.

We now can try to separate the terms of the loss function so that we can look at just one group l at a time:

$$f(\beta_l) = \frac{1}{2}\|R_l - X_l\beta_l\|_2^2 + \lambda\sqrt{p_l}\|\beta_l\|_2 + C$$

We are looking at solution for β_l so we ignore the terms that do not depend on it and leave those terms in C.

Now let us simplify and try to compute the gradient. Remember $X_l^T X_l = \mathbf{I}$. Then:

$$\frac{1}{2}(R_l - X_l\beta_l)^T(R_l - X_l\beta_l) = \frac{1}{2}(R_l^T R_l - 2\beta_l^T X_l^T R_l + \beta_l^T X_l^T X_l\beta_l) = \frac{1}{2}R_l^T R_l - \beta_l^T(X_l^T R_l) + \frac{1}{2}\|\beta_l\|_2^2$$

Remember $S_l = X_l^T R_l = X_l^T(Y - \sum_{k\neq l} X_k\beta_k)$. We can simplify our function further:

$$\frac{1}{2}\|\beta_l\|_2^2 - \beta_l^T S_l + \lambda\sqrt{p_l}\|\beta_l\|_2$$

This is the function we want to minimize with respect to β_l:

$$\min_{\beta_l} \left\{ \frac{1}{2}\|\beta_l\|_2^2 - \beta_l^T S_l + \lambda\sqrt{p_l}\|\beta_l\|_2 \right\}$$

We take the derivative and set to 0:

$$\frac{\partial}{\partial \beta_l}\left(\frac{1}{2}\beta_l^T\beta_l - \beta_l^T S_l + \lambda\sqrt{p_l}\|\beta_l\|_2\right) = \beta_l - S_l + \lambda\sqrt{p_l}\frac{\beta_l}{\|\beta_l\|_2} = 0$$

We solve for β_l:

$$\beta_l\left(1 + \frac{\lambda\sqrt{p_l}}{\|\beta_l\|_2}\right) = S_l$$

We try to simplify. We can get rid of a β_l denominator if we take the norm of both sides:

$$\left\|\hat{\beta}_l\left(1 + \frac{\lambda\sqrt{p_l}}{\|\hat{\beta}_l\|_2}\right)\right\|_2 = \|\hat{\beta}_l\|_2 \cdot \left(1 + \frac{\lambda\sqrt{p_l}}{\|\hat{\beta}_l\|_2}\right) = \|S_l\|_2$$

Then:

$$\|\hat{\beta}_l\|_2 + \lambda\sqrt{p_l} = \|S_l\|_2 \implies \|\hat{\beta}_l\|_2 = \|S_l\|_2 - \lambda\sqrt{p_l}$$

Ok, we have an identity. We go back to original derivative: $\beta_l\left(1 + \frac{\lambda\sqrt{p_l}}{\|\beta_l\|_2}\right) = S_l$. We re-arrange and plug in our solution:

$$\hat{\beta}_l = \frac{S_l}{1 + \frac{\lambda\sqrt{p_l}}{\|S_l\|_2 - \lambda\sqrt{p_l}}} = S_l\left(\frac{\|S_l\|_2 - \lambda\sqrt{p_l}}{\|S_l\|_2}\right) = \left(\mathbf{1} - \frac{\lambda\sqrt{\mathbf{p_l}}}{\|\mathbf{S_l}\|_\mathbf{2}}\right)\mathbf{S_l}$$

(Simplification of denominator: $1 + \frac{\lambda\sqrt{p_l}}{\|S_l\|_2 - \lambda\sqrt{p_l}} = \frac{\|S_l\|_2 - \lambda\sqrt{p_l} + \lambda\sqrt{p_l}}{\|S_l\|_2 - \lambda\sqrt{p_l}} = \frac{\|S_l\|_2}{\|S_l\|_2 - \lambda\sqrt{p_l}}$)

Problem 2

Consider integer-valued i.i.d. data $Y_1, \ldots, Y_n$ from a truncated Poisson distribution with pmf:

$$p(y;\lambda) = \begin{cases} \frac{e^{-\lambda}\lambda^y}{y!\,(1-e^{-\lambda}-\lambda e^{-\lambda})}, & \text{for } y = 2, 3, \ldots \\ 0, & \text{otherwise.} \end{cases}$$

Set the seed in R so your results can be replicated. Set true $\lambda = 5$. Use a simple rejection sampling scheme to simulate 100 observations from the truncated Poisson distribution. Now, derive and implement a gradient descent algorithm to find the MLE with starting value $\lambda_0 = 1$. Prepare a table or figure of log-likelihood values against iteration number until the search terminates. Use a tolerance of 10^{-5}.

To begin with, we have poisson distribution truncated at $y \leq 2$. We are given the PMF. We find the log-likelihood:

$$\begin{aligned} \ell(\lambda) &= \sum_{i=1}^{n} \left[-\lambda + y_i \ln(\lambda) - \ln(y_i!) - \ln(1 - e^{-\lambda} - \lambda e^{-\lambda})\right] \\ &= -n\lambda + \ln(\lambda) \sum y_i - \sum \ln(y_i!) - n \ln(1 - e^{-\lambda} - \lambda e^{-\lambda}) \end{aligned}$$

We take the derivative with respect to λ. Note $\frac{d}{d\lambda}(1 - e^{-\lambda} - \lambda e^{-\lambda}) = e^{-\lambda} - (e^{-\lambda} - \lambda e^{-\lambda}) = \lambda e^{-\lambda}$. We find:

$$\ell'(\lambda) = -n + \frac{\sum y_i}{\lambda} - n\frac{d}{d\lambda}\ln(1 - e^{-\lambda} - \lambda e^{-\lambda}) = -n + \frac{\sum y_i}{\lambda} - n\frac{\lambda e^{-\lambda}}{1 - e^{-\lambda} - \lambda e^{-\lambda}}$$

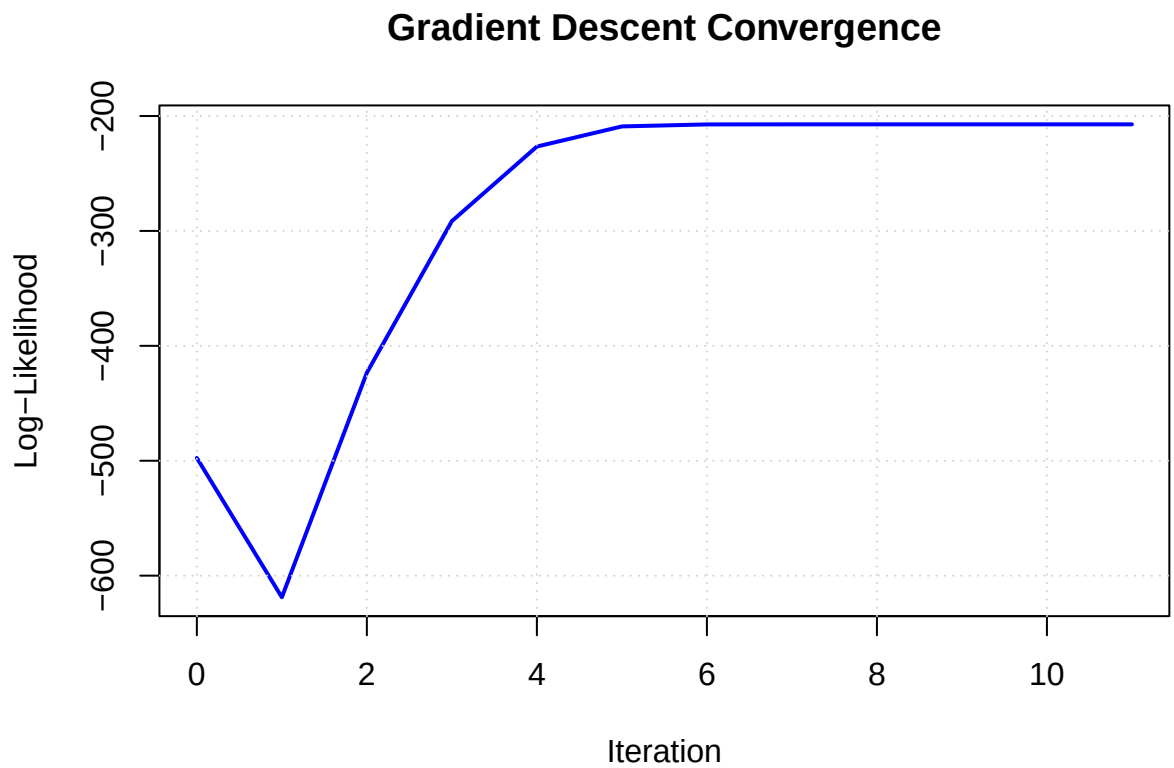
This is technically all we need for simple gradient descent algorithm. We use same seed as last week. Remember that we are maximizing, so we go up our gradient. We implement a simple steepest descent method of gradient descent (ascent technically).

```r
set.seed(123)
true_lambda <- 5
n <- 100

# 1. Really Basic Rejection Sampling Function
# Just use R function for poisson
simulate_truncated_poisson <- function(n, lambda) {
  samples <- numeric(0)
  while(length(samples) < n) {
    # Propose from a standard Poisson
    proposal <- rpois(n, lambda)
    # Reject values 0 and 1
    valid <- proposal[proposal >= 2]
    samples <- c(samples, valid)
  }
  return(samples[1:n])
}

y <- simulate_truncated_poisson(n, true_lambda)

# 2. Log-likelihood and Gradient Functions
log_lik <- function(lambda, y) {
```

```r
  n <- length(y)
  -n * lambda + log(lambda) * sum(y) - sum(lfactorial(y)) - n * log(1 - exp(-lambda) - lambda * exp(-lam
}
gradient <- function(lambda, y) {
  n <- length(y)
  -n + sum(y)/lambda - n * (lambda * exp(-lambda)) / (1 - exp(-lambda) - lambda * exp(-lambda))
}

# 3. Gradient Descent
lambda_curr <- 1 # Starting value
alpha <- 0.05   # Learning rate, 0.05 seems to converge quite fast
tol <- 1e-5 # Tolerance
max_iter <- 2000 # Set a hard limit for iteration also
history <- data.frame(iter = 0, lambda = lambda_curr, loglik = log_lik(lambda_curr, y))

for (i in 1:max_iter) {
  grad <- gradient(lambda_curr, y)
  lambda_next <- lambda_curr + alpha * grad # We add because we maximize so go up the gradient

  new_loglik <- log_lik(lambda_next, y)
  history <- rbind(history, c(i, lambda_next, new_loglik))

  if (abs(lambda_next - lambda_curr) < tol) {
    break
  }
  lambda_curr <- lambda_next
}

# Results
print(paste("Final MLE for lambda:", round(lambda_curr, 5)))
```

```
## [1] "Final MLE for lambda: 4.9034"
```

```r
# Simple plotting the log-likelihood over iterations
plot(history$iter, history$loglik, type = "l", col = "blue", lwd = 2,
     xlab = "Iteration", ylab = "Log-Likelihood",
     main = "Gradient Descent Convergence")
grid()
```




```r
summary(history)
```

```
##       iter           lambda           loglik
##  Min.   : 0.00   Min.   : 1.000   Min.   :-618.8
##  1st Qu.: 2.75   1st Qu.: 4.903   1st Qu.:-324.5
##  Median : 5.50   Median : 4.945   Median :-208.2
##  Mean   : 5.50   Mean   : 6.383   Mean   :-292.5
##  3rd Qu.: 8.25   3rd Qu.: 6.995   3rd Qu.:-207.2
##  Max.   :11.00   Max.   :14.489   Max.   :-207.2
```

```r
history
```

```
##    iter    lambda    loglik
## 1     0  1.000000 -497.5651
## 2     1 14.488944 -618.7946
## 3     2 11.245419 -423.4202
## 4     3  8.507828 -291.4828
## 5     4  6.490586 -226.5601
## 6     5  5.361831 -209.0403
## 7     6  4.978676 -207.2912
## 8     7  4.911778 -207.2413
## 9     8  4.904241 -207.2407
## 10    9  4.903476 -207.2406
## 11   10  4.903399 -207.2406
## 12   11  4.903391 -207.2406
```